

Reading Data Sources

Silent defaults in `read_csv` are the #1 source of production data bugs. By the end of this lesson, no default will surprise you again.

Have you ever seen "007" become 7 in a CSV? Show of hands. We fix that today.

WHAT'S INSIDE

- 01** CSV — the lingua franca and its traps
- 02** Excel — the CFO's native format
- 03** SQL — where enterprise data lives
- 04** JSON — API responses and logs
- 05** Write-back — same API out as in

WHY THIS LESSON MATTERS

Four file formats. Four different traps. One day of your life saved.

WHY

Real

A real reason this matters in industry today.

SCALE

Today

A real reason this matters in industry today.

SPEED

Now

A real reason this matters in industry today.

IMPACT

Yours

A real reason this matters in industry today.

LEARNING OUTCOMES

By the end of this lesson, you will be able to —

1

Read a CSV with correct encoding, separators, and dtypes.

2

Read a multi-sheet Excel file with pandas.

3

Query a SQLite database with `pd.read_sql`.

4

Parse nested JSON with `json_normalize`.

5

Round-trip a DataFrame through all four formats.

AGENDA

What we'll cover today

Short, sequenced parts. Each one builds on the last.

01

CSV — the lingua franca and...

Read with explicit parameters, always

02

Excel — the CFO's native...

Multi-sheet, header-skip, cell-range

03

SQL — where enterprise data...

Push filters into SQL, not pandas

04

JSON — API responses and...

Flat vs nested

01

PART 1 OF 4

CSV — the lingua franca and...

Read with explicit parameters, always

CONCEPT 1 OF 4

CSV — the lingua franca and its traps

encoding="utf-8-sig" — Indian characters + BOM handling.
parse_dates=["order_date"] — dates parsed, not strings.
dtype={"id": str} — preserve leading zeros.
na_values=["--", "N/A", "null"] — catch non-standard missing.
chunksize=100_000 — stream large files in chunks.

RULE

No silent defaults

Every read_csv call should be intentional.

DEFINITION

CSV — the lingua franca and its traps

Read with explicit parameters, always

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

CONCEPT 2 OF 4

Excel — the CFO's native format

`pd.read_excel("s.xlsx", sheet_name=None)` → dict of DataFrames.
`pd.read_excel("s.xlsx", sheet_name="Q1", header=2)` — skip title rows.
`usecols="A:D"` — column range.
`to_excel("out.xlsx", index=False)` — no row index.

AUDIENCE

Finance teams

Excel in, Excel out. Python is the bridge.

KEY POINTS

- `pd.read_excel("s.xlsx", sheet_name=None)` → dict of DataFrames
- `pd.read_excel("s.xlsx", sheet_name="Q1", header=2)` — skip title rows
- `usecols="A:D"` — column range
- `to_excel("out.xlsx", index=False)` — no row index

CONCEPT 3 OF 4

SQL — where enterprise data lives

```
con = sqlite3.connect("shops.sqlite")  
df = pd.read_sql("SELECT * FROM orders WHERE city='Bangalore'", con)  
df.to_sql("out", con, index=False, if_exists="replace")
```

Large tables: ALWAYS WHERE-clause early.

RULE

Filter in SQL

50k filtered rows beat 50M read-then-filtered, every time.

DEFINITION

SQL — where enterprise data lives

Push filters into SQL, not pandas

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

JSON — API responses and logs

Flat: `pd.read_json("orders.json")`.

Nested: `from pandas import json_normalize`.

`json_normalize(data, record_path="items", meta="order_id")` — one row per item.

Never hand-loop to flatten. Use `json_normalize`.

STAR

`json_normalize`

Two-week bugs disappear when you use the right tool.

KEY POINTS

- Flat: `pd.read_json("orders.json")`
- Nested: `from pandas import json_normalize`
- `json_normalize(data, record_path="items", meta="order_id")` — one row per item
- Never hand-loop to flatten. Use `json_normalize`

02

PART 2 OF 4

Engage and reflect

Pause, talk, and connect what you just learned to your own work.

✦ THINK • PAIR •
SHARE

ACTIVITY

Load the same data from all four formats

PROMPT

Apply the four concepts above to one real example from your own week.

HOW TO RUN IT

1

2

3

4

Reveal: instructor confirms the canonical answer.

REFLECT & APPLY

Three questions before you close this lesson

Spend 60 seconds on each. Write the answers down — no scrolling, no shortcuts.

Q1

Which of the four formats have you actually used in anger?

Q2

Where did a silent default bite you in the past?

Q3

Which format will be most common in your first real job?

03

PART 3 OF 4

Knowledge check

Three short questions. One minute each. Honest answers only.

To fix Indian characters garbled in CSV, set:

A

sep=";"

B

encoding="utf-8-sig"

C

dtype=str

D

header=0

To fix Indian characters garbled in CSV, set:



CORRECT ANSWER

`encoding="utf-8-sig"`

WHY

utf-8-sig handles Indian character encoding + BOM correctly

pd.read_excel(sheet_name=None) returns:

A

DataFrame

B

Series

C

dict of DataFrames

D

list

pd.read_excel(sheet_name=None) returns:



CORRECT ANSWER

dict of DataFrames

WHY

None as the sheet_name returns a dictionary of {sheet: DataFrame}

A 50M-row SQL table needs filtering to one city. Where?

A

Read all; filter in pandas

B

Filter in SQL

C

Read in chunks

D

Skip SQL

A 50M-row SQL table needs filtering to one city. Where?



CORRECT ANSWER

Filter in SQL

WHY

Push the WHERE clause into SQL — 100× faster than in-memory filtering

04

PART 4 OF 4

Apply and close

One thing to remember. One thing to ship this week.

SUMMARY

Five sentences to remember

If you remember nothing else from this lesson, remember these five lines.

- 1 Pick the right tool — never the loudest one.
- 2 Practice the framework on a real example.
- 3 Watch for the three classic pitfalls.
- 4 Document your decision in one sentence.
- 5 Carry the discipline forward to the next lesson.

WORKED EXAMPLE

How this lesson plays out in one real Indian context

THE SCENARIO

encoding="utf-8-sig" — Indian characters + BOM handling.
parse_dates=["order_date"] — dates parsed, not strings.
dtype={"id": str} — preserve leading zeros.
na_values=["--", "N/A", "null"] — catch non-standard missing.
chunksize=100_000 — stream large files in chunks.

RULE

No silent defaults

Every read_csv call should be intentional.

THE TAKEAWAY

"One DataFrame, four interfaces. The data scientist who masters all four moves fastest."

WHAT TO NOTICE

1

CSV — the lingua franca and...

2

Excel — the CFO's native...

3

SQL — where enterprise data...

4

JSON — API responses and...

Mini assignment — round-trip one dataset

Pick any dataset. Write it out to all four formats; read all four back; assert they produce identical DataFrames.

DELIVERABLES

- Start with a cleaned DataFrame from Lesson 4.
- Write .csv, .xlsx, .sqlite, .json.
- Read each back into a new DataFrame.
- `pd.testing.assert_frame_equal` between each pair.

WHAT 'GOOD' LOOKS LIKE

Deliverable: Notebook URL + screenshot of all four asserts passing.

ONE THING TO REMEMBER

“

"One DataFrame, four interfaces. The data scientist who masters all four moves fastest."

— AI Learning Hub

END OF LESSON 2

What you can now do

Each one of these is now part of your working vocabulary.

- 1 Pick the right tool — never the loudest one.
- 2 Practice the framework on a real example.
- 3 Watch for the three classic pitfalls.
- 4 Document your decision in one sentence.
- 5 Carry the discipline forward to the next lesson.

NEXT → Project — Indian E-commerce Wrangling. All four formats, one analysis, 90...